

Model Collapse in Tabular Generation Models

Yuxiang Wei Xingxiang Huang Warren Wu Jared Gode

December 12, 2025

Abstract

We investigate model collapse in tabular data generation, inspired by recent findings of collapse in large language models trained on synthetic text. Using a highly imbalanced click-through-rate dataset (roughly 1.5% positives), we evaluate three generative paradigms—CTGAN, CVAE, and the autoregressive LLM-based GReaT—under an iterative retraining regime where each synthesizer is fine-tuned on a 50/50 mixture of real and self-generated samples. Across all models, we observe consistent forms of collapse, including overinflation of minority-class modes and systematic distortion of feature–feature correlation structure.

CTGAN variants oversample the minority class and fail to preserve global relationships, though CTGAN-on-CTGAN training can still improve downstream predictive performance. CVAE V1 underperforms real-data baselines, while CVAE V2 improves downstream accuracy but introduces spurious correlations between unrelated features. The LLM-based generator hallucinates categorical and label values, producing the lowest fidelity among all methods despite stable downstream performance across iterations.

Our results show that while synthetic-on-synthetic training can enhance downstream models, it simultaneously degrades statistical fidelity, indicating that tabular generative models are vulnerable to collapse effects similar to those seen in LLMs. This highlights the need for collapse-aware evaluation protocols when using synthetic data for machine-learning pipelines.

1 Introduction

Synthetic data generation has become an increasingly important tool for addressing data scarcity, privacy concerns, and distributional shift in machine-learning pipelines. Recent work has shown, however, that generative models, particularly large language models (LLMs), can undergo model collapse when repeatedly trained on their own outputs, leading to degraded diversity, distorted distributions, and hallucinated structure [Shumailov et al., 2024]. While this phenomenon is well studied in natural language settings, its behavior in tabular domains remains underexplored. In this project, we investigate whether common tabular data synthesizers exhibit analogous collapse effects when repeatedly exposed to synthetic rather than real training data.

Our experiments are conducted on a large-scale click-through-rate (CTR) prediction dataset originating from an online advertising system. The dataset contains a mixture of

user, device, application, and interaction features alongside a binary label indicating whether an ad was clicked. A defining characteristic of this domain is its extreme class imbalance: only about 1.5% of samples correspond to positive (clicked) events. This imbalance creates challenges for both discriminative and generative modeling; effective models must capture not only the dominant negative class but also the rare minority patterns that drive meaningful downstream performance.

Before studying generative collapse, we establish a strong predictive baseline using gradient-boosted decision trees. Consistent with industrial practice, LightGBM achieves the highest performance among all supervised models we tested, substantially outperforming logistic regression, MLPs, and other classical baselines. LightGBM therefore serves both as (i) a sanity check for the quality and fidelity of each synthetic dataset, and (ii) the downstream task model used to assess whether synthetic data helps or harms the CTR prediction task.

On the generative side, we evaluate three representative families of tabular synthesizers: CTGAN, CVAE, and the autoregressive GReaT framework based on fine-tuned GPT-2. We then study collapse by retraining each synthesizer on a 50% real / 50% self-generated dataset and measuring how distributional fidelity, feature–feature correlations, and downstream predictive performance degrade across iterations. Across all models, we observe systematic distortions—such as overinflation of minority-class modes, hallucinated categorical values, and spurious correlations—that parallel the collapse behaviors documented in LLM literature.

Our goal is to characterize how robust (or fragile) modern tabular generative models are under iterative synthetic-on-synthetic training and to highlight evaluation practices necessary to detect collapse before synthetic data is deployed in real ML workflows.

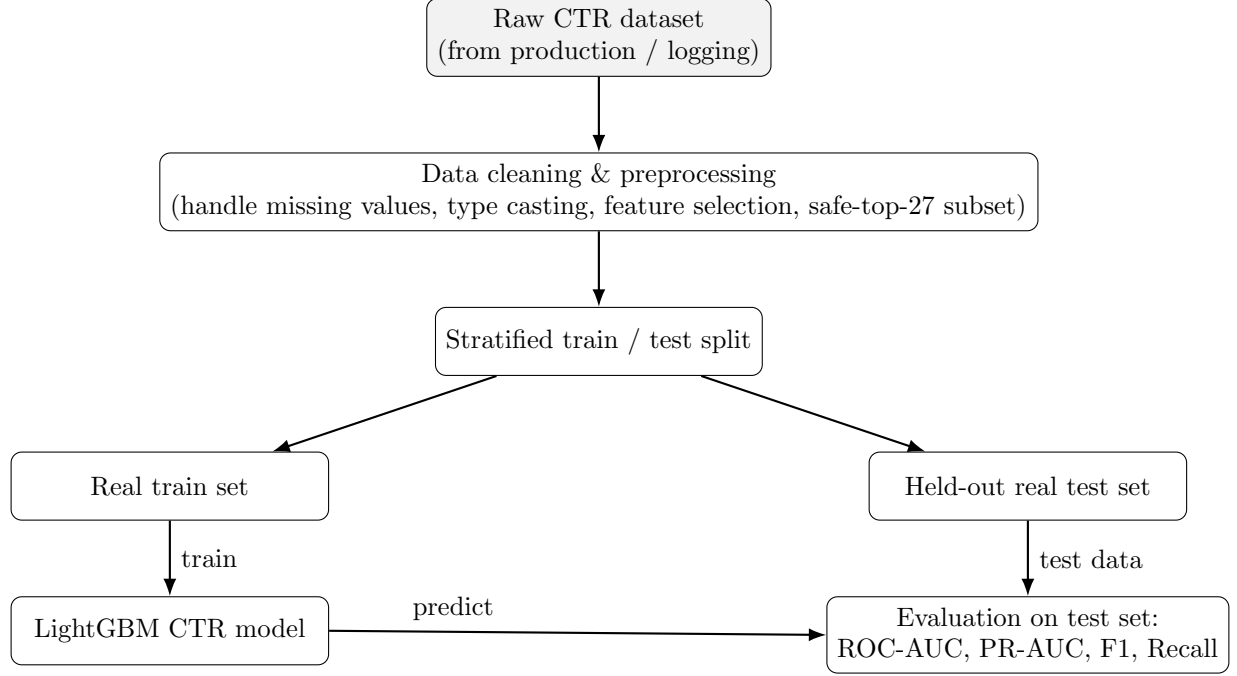


Figure 1: Baseline CTR prediction pipeline. Starting from raw logged data, we first perform data cleaning and preprocessing (e.g. missing-value handling, type casting, selection of safe features), then split into train/test sets, train LightGBM on the real train set, and evaluate on a held-out real test set.

Using LightGBM as Downstream Evaluation Model, using ROC-AUC and F1 as Evaluation metric

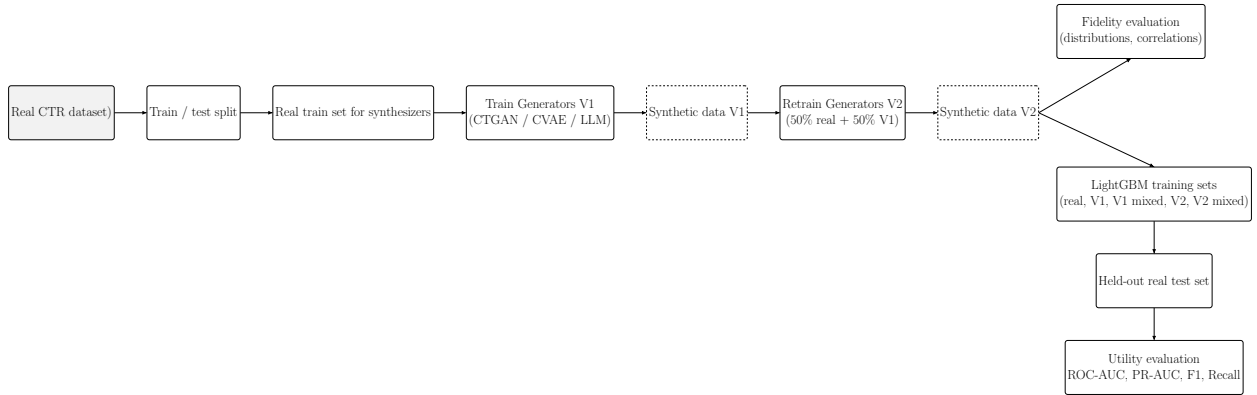


Figure 2: Horizontal synthetic-data pipeline. Real CTR data is used to train V1 generators (CTGAN, CVAE, LLM), produce Synthetic V1, retrain V2 generators, and generate Synthetic V2. From V2, two branches evaluate fidelity (right-up) and downstream utility via LightGBM (right-down).

2 Data Preprocessing

2.1 Data Overview

The study uses the dataset released by the 2022 DIGIX Global AI Challenge. It contains two complementary domains: an ads domain used for click-through-rate prediction and a feeds domain that records users’ browsing behavior in a news-feed environment. The ads tables provide impression-level information and the binary click label, while the feeds tables offer richer signals on user interests, activity patterns, and device attributes. Four raw files—training and test data for each domain—form the basis of the preprocessing stage.

2.2 Data Cleaning

The ads and feeds datasets were first standardized to ensure consistent structure, valid types, and reliable timestamps. Missing numeric values were assigned a default value of -1 , while missing categorical fields were replaced with an “unknown” token. Multi-value string fields were normalized by collapsing irregular separators into a single delimiter.

Data types were aligned across files: identifier columns were cast to categorical types, and all continuous variables were converted to integers or floats to maintain numerical comparability. Temporal fields in both datasets were parsed from raw timestamps. From these, three features were extracted—hour of day, weekday, and a binary weekend indicator—providing a consistent temporal reference across ads and feeds logs.

These steps ensured structural integrity between training and test splits and produced clean tables suitable for downstream merging and feature construction.

2.3 User-Level Aggregation

To enrich the ads dataset with behavioral information, the feeds logs were aggregated into one row per user. All aggregations were grouped by `u_userId`. The resulting profile summarizes five aspects of user behavior:

- **Activity level:** feed count (`f_rows`), refresh frequency (`f_refresh_mean`, `f_refresh_sum`).
- **Interest breadth:** unique news categories (`f_cat_uniq`), average entity count (`f_entities_len_mean`).
- **Feedback behavior:** upvotes (`f_up_mean`, `f_up_sum`) and dislikes (`f_dislike_mean`, `f_dislike_sum`).
- **Temporal preference:** average and peak browsing hours (`f_avg_hour`, `f_peak_hour`).
- **Device traits:** median phone price and most frequent browser mode.

To model circadian patterns more smoothly, the average browsing hour was transformed into two cyclic encodings:

$$\text{hour_sin} = \sin\left(\frac{2\pi \cdot \text{avg_hour}}{24}\right), \quad \text{hour_cos} = \cos\left(\frac{2\pi \cdot \text{avg_hour}}{24}\right).$$

This representation avoids artificial discontinuities at the 24-hour boundary and helps tree-based models capture periodic behaviors.

A light exploratory check confirmed expected distributions: activity counts were long-tailed, browsing peaks centered around evening hours, and device prices were stable across users. The aggregated tables were exported for merging with the ads records.

2.4 Dataset Merge

We merged the ad-level dataset with the aggregated user-behavior features from feeds. The merge key is `user_id = u.userId`, applied to both training and test sets. After dropping duplicated ID columns and verifying type consistency, the merged datasets reached:

- Training set: $\sim 7.68\text{M rows} \times 68\text{ columns}$
- Test set: $\sim 0.98\text{M rows} \times 67\text{ columns}$

Basic checks confirmed stable CTR distribution and reasonable feature ranges. This step enriches each ad impression record with user-level information.

2.5 Exploratory Data Analysis (EDA)

Pre- and post-merge explorations were conducted to understand the structure of the dataset and the relationships between features and the click-through label.

We first examined the categorical fields in the ads dataset by converting them into dummy variables and computing their correlations with the CTR label. None of the encoded variables showed meaningful association with the response; the largest absolute correlations were around ± 0.08 . This indicates that these identifiers contribute little linear signal on their own.

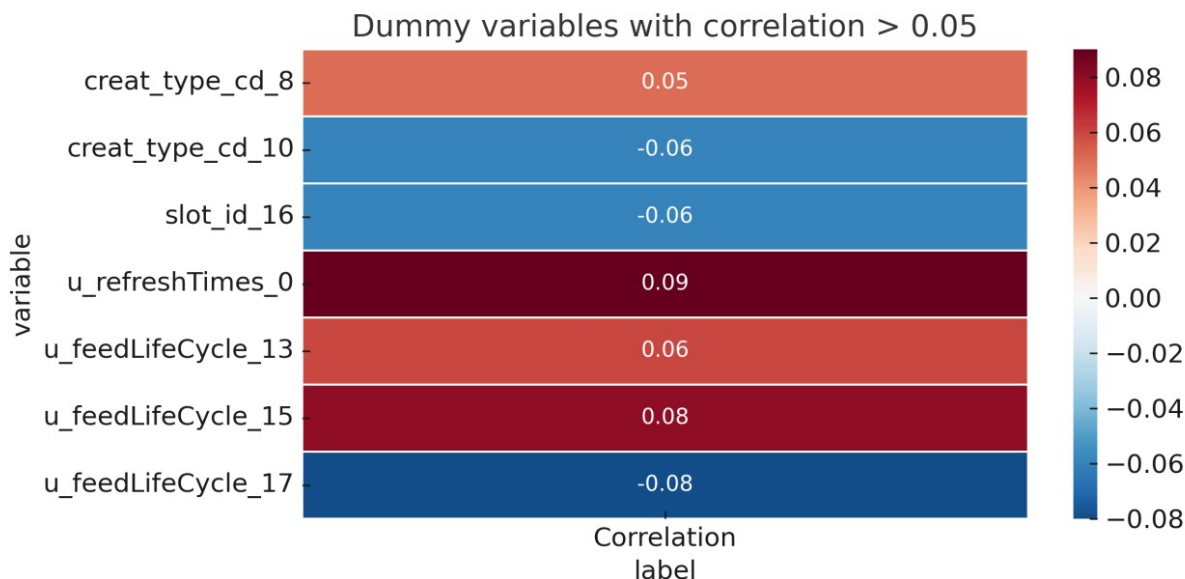


Figure 3: Top dummy variable correlations with the CTR label

CTR distribution and demographic patterns. Before merging, CTR was highly imbalanced at approximately 1.55%. CTR varied across age groups, was slightly lower for female users, and displayed clear temporal trends, with peaks in early morning and late evening.

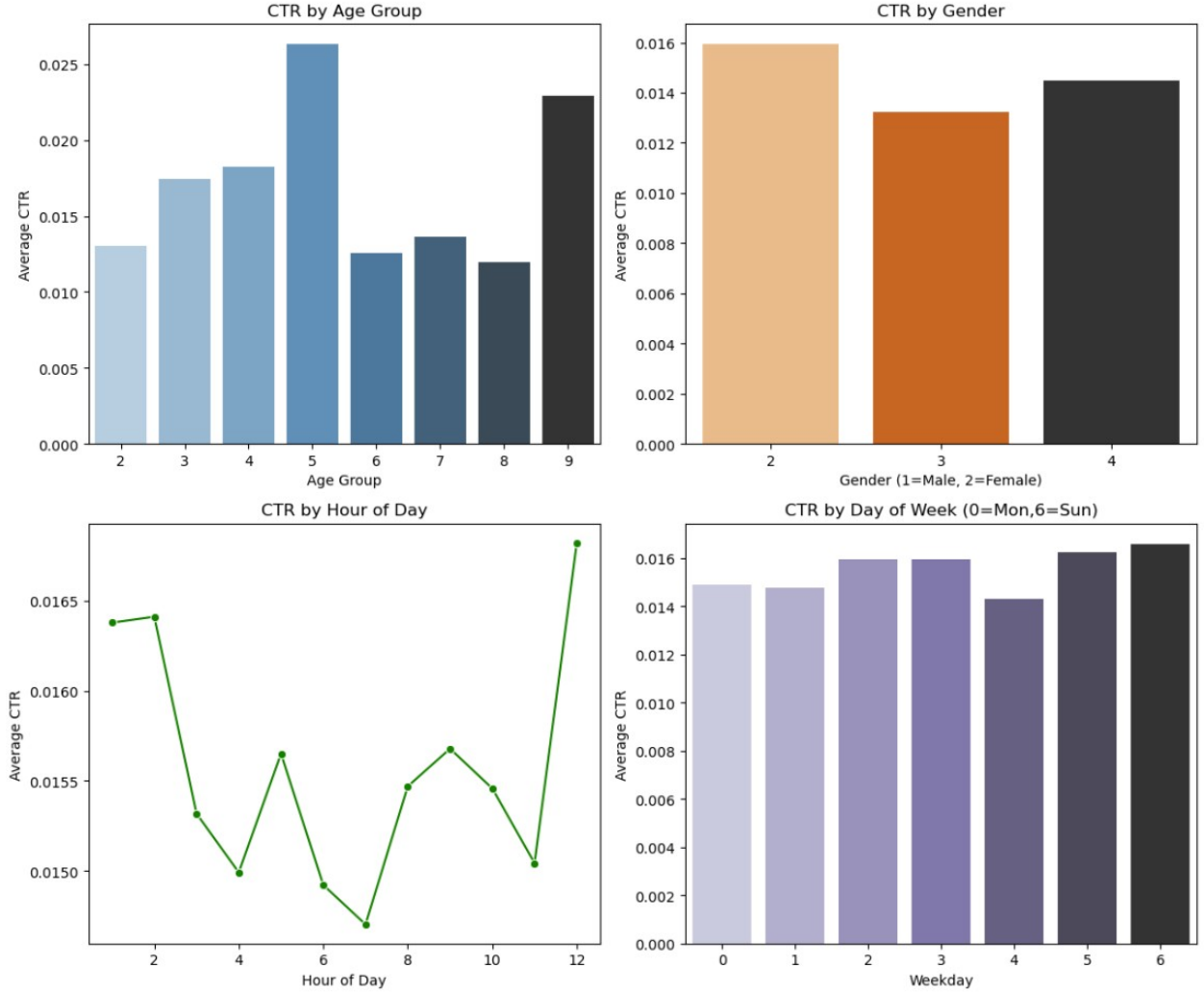


Figure 4: CTR by demographics and time.

Post-merge correlation structure. Behavioral features introduced new explanatory signals—variables such as `f_up_mean`, `f_dislike_mean`, and `f_hour_cos` showed mild but consistent correlations with CTR. The full correlation heatmap highlighted several stable feature clusters, including strong internal correlations among length-based and count-based variables, and weaker correlations between behavioral features and demographic fields.

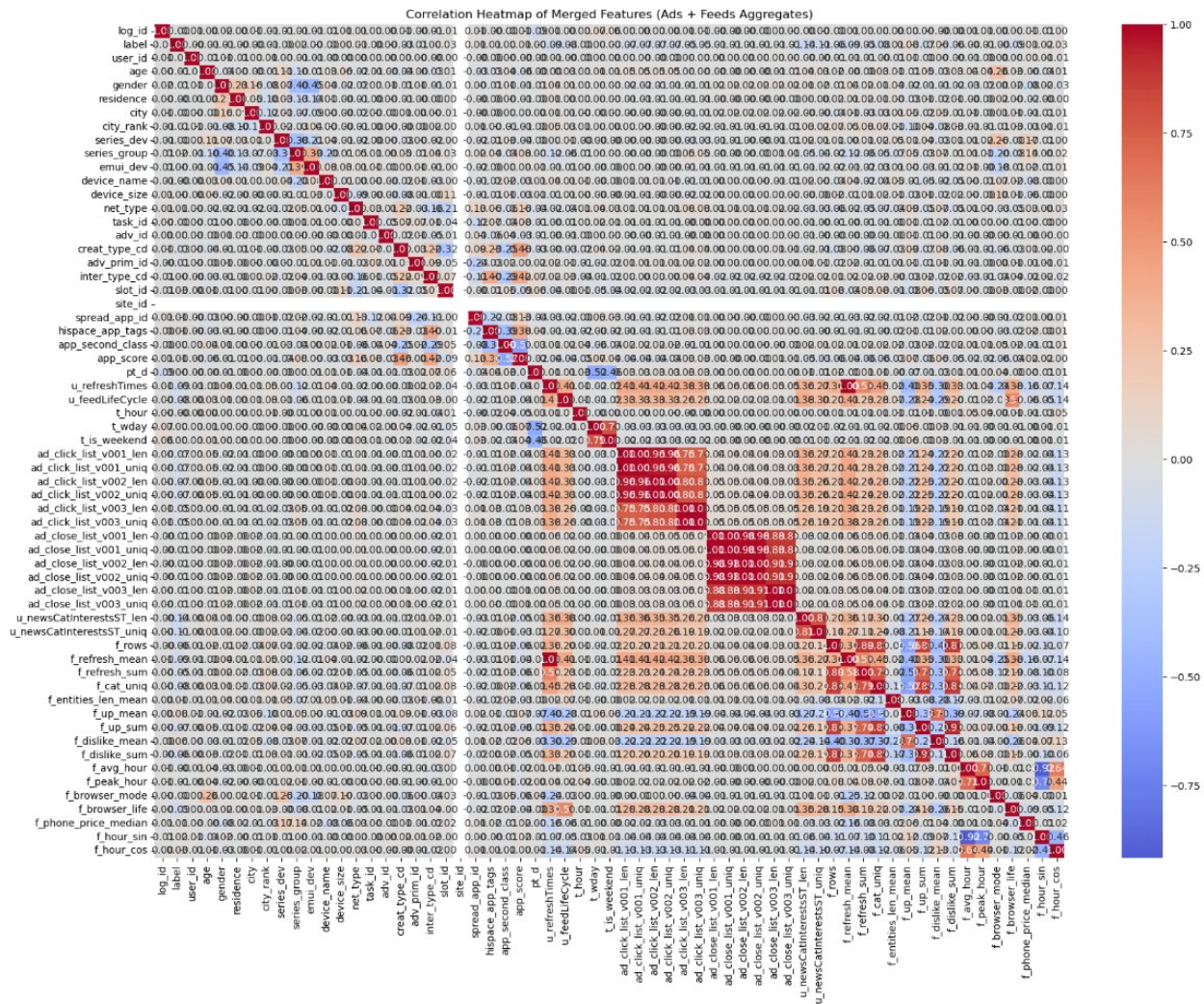


Figure 5: Correlation heatmap of merged user and ad features.

Feature distributions by click label. Boxplots of behavioral variables revealed distinct patterns between clicked and non-clicked impressions.

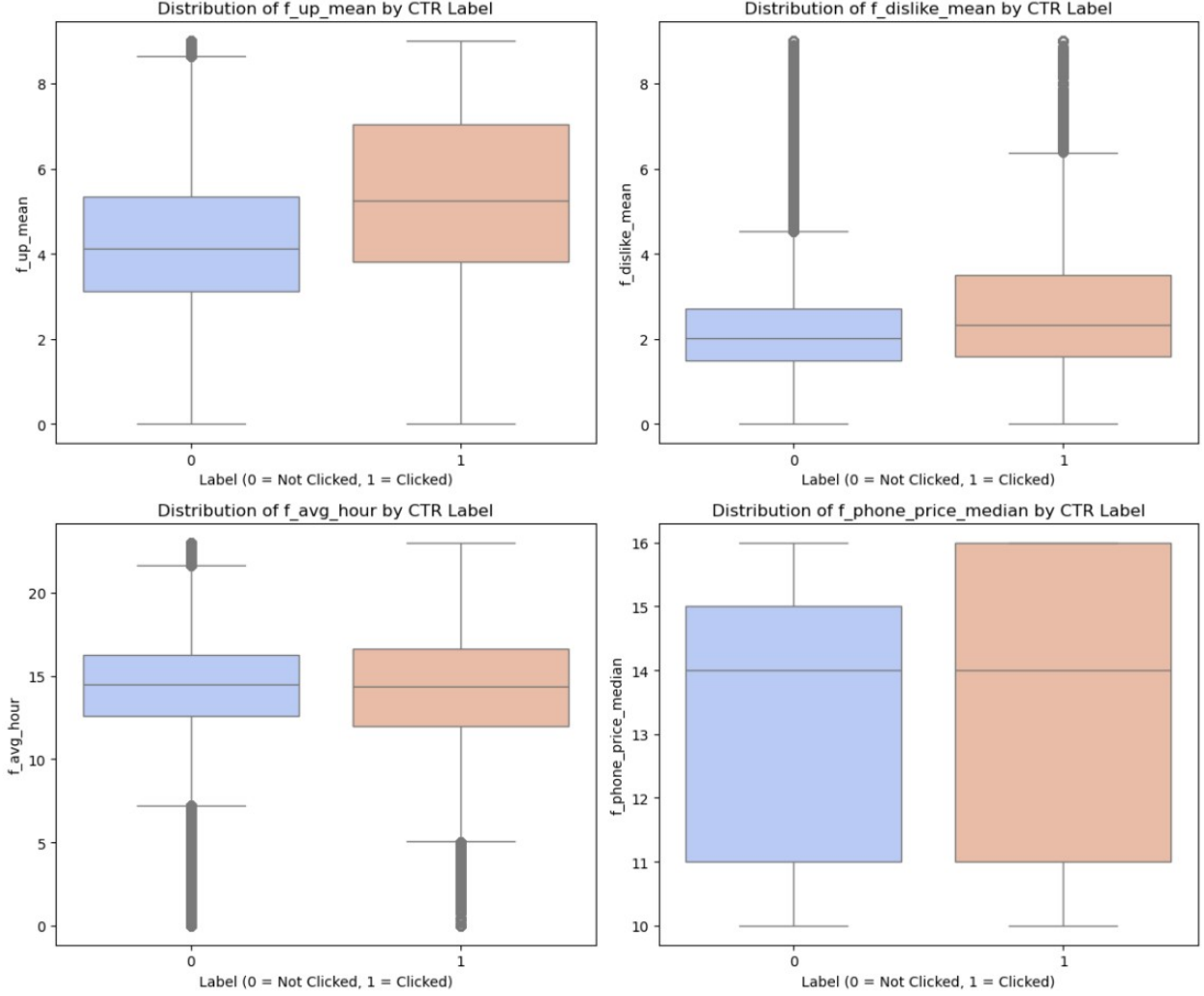


Figure 6: Distributions of key behavioral features for clicked vs. non-clicked samples.

Overall, the merged dataset showed meaningful behavioral variation and consistent structure suitable for model training.

2.6 Feature Selection and Safe Top-27 Set

Beyond these descriptive checks, we computed LightGBM feature-importance scores to screen for potential leakage. Several high-gain variables such as `log_id`, `user_id`, and `pt_d` were identified as sample or user identifiers rather than meaningful behavioral signals. Because such fields may inflate performance or compromise fairness, they were removed from the feature pool.

The remaining twenty-seven behavioral and contextual variables constitute the Safe Top-27 feature set, which is used consistently in all subsequent modeling and synthetic-data evaluations.

2.7 Baseline CTR Model (LightGBM)

To establish a consistent downstream evaluation pipeline, we trained a click-through-rate (CTR) prediction model using the curated Top-27 numerical feature set together with the binary click label. The dataset was split into training and test portions using a stratified 80/20 partition to maintain the original class imbalance.

Several standard models were explored, including logistic regression, random forest, and gradient-boosted decision trees. Among them, LightGBM delivered the strongest discrimination performance and was therefore adopted as the baseline model for all subsequent utility evaluations.

The ROC curve demonstrates clear separation between positive and negative samples despite the 1–2% CTR. The precision–recall curve reflects the difficulty of the task: although absolute precision remains low, the model provides substantial improvement over the random baseline.

Table 1: Baseline LightGBM Performance

Metric	Value
ROC-AUC	0.812
PR-AUC	0.139
F1	0.098
Precision	0.053
Recall	0.626

These metrics align with the behavior expected for large-scale CTR prediction tasks: LightGBM provides stable ranking ability (high ROC-AUC), modest PR-AUC under severe imbalance, and low F1 due to the rarity of positive samples.

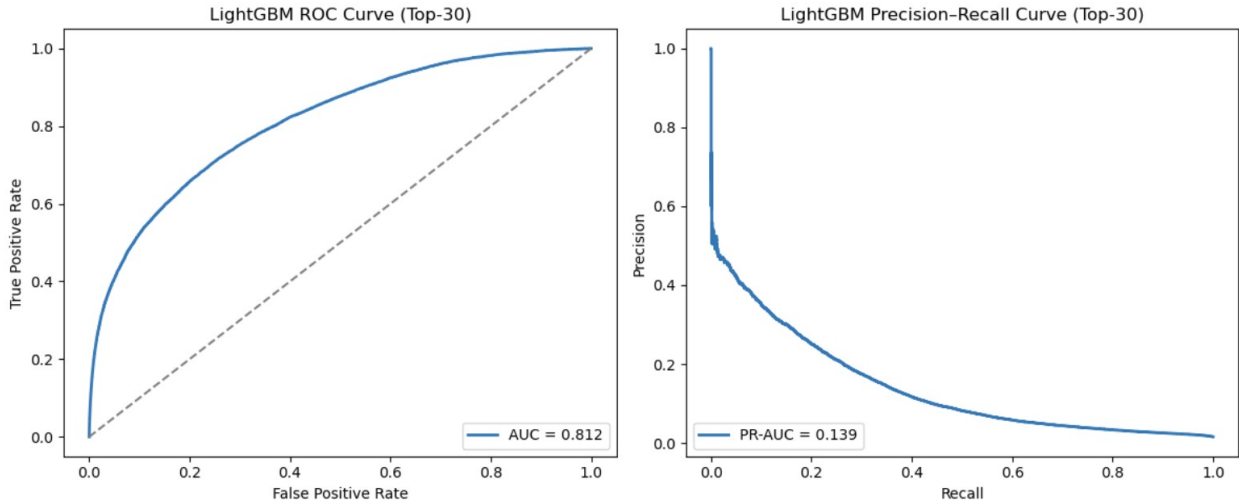


Figure 7: Baseline LightGBM performance: ROC and Precision–Recall curves.

This baseline LightGBM model is reused throughout the paper to evaluate the utility of the synthetic datasets.

3 Methods

3.1 Synthesizer V1

The first synthesizer for each of the three methods we explored was trained exclusively on real data, taken in subsets from the click-through rate data set in order to make training time manageable with limited computational power. This synthesizer model then produced a synthetic data set, which was used to train a LightGBM model for later testing.

3.2 Synthesizer V2

The second synthesizer for our three methods was trained on a mix of real and synthetic data, the real data again taken from the click through rate data set, and the synthetic data taken from what was generated by our V1 synthesizers. These new synthesizers then produced a synthetic data set, which was then put to use when training a LightGBM model for further testing.

3.3 Evaluation

The LightGBM models trained with the synthetic data were used to gather utility statistics. These synthetic LightGBM models were then compared against LightGBM models trained with entirely real data and models made with partially real and partially synthetic data. Beyond this, the pairwise correlation of the synthetic datasets was compared to the pairwise correlation of the training data to ensure that the structure and relationships present in the original data were shared by the synthetic data.

4 CTGAN

4.1 Training Procedure

CTGAN [Xu et al., 2019] is adopted as the first generative model for synthesizing CTR data due to its suitability for mixed-type tabular inputs and strong class imbalance. The training dataset is constructed from the Safe Top-27 numerical features together with the binary CTR label. All user IDs, timestamps, and other leakage-prone variables are removed so that the model learns general behavioural structure rather than unique identifiers.

We reload the cleaned merged dataset and reconstruct the stratified 80/20 train-validation split used in earlier analyses. A total of about 6.14 million rows form the training pool. From this pool, a 200,000-row subset is sampled for CTGAN training, balancing computational feasibility and distributional richness. During preprocessing, 18 feature columns are automatically detected as discrete based on their cardinality patterns, including fields such as `creat_type_cd`, `slot_id`, `inter_type_cd`, and several app/browser categorical codes.

CTGAN is trained using a lightweight configuration (10 epochs, batch size 1024, `pac` = 1, hidden dimensions 256–256), which allows stable convergence within minutes while maintaining sufficient expressive power. After training, the generator is used to sample 200,000 synthetic records following the same schema as the input features.

4.2 Synthetic Data Characteristics

The resulting synthetic dataset preserves the Safe Top-27 feature schema and produces a synthetic CTR rate of approximately 5.11%, which is substantially higher than the real CTR rate near 1.55%. This is a known behaviour of GANs trained on extremely imbalanced binary outcomes: the generator tends to oversample positive events, reflecting mode collapse along the minority-class dimension.

A preview of the generated synthetic rows (not shown here for brevity) confirms that feature ranges and value types are plausible.

4.3 Utility Evaluation

We assess the usefulness of CTGAN-generated data by training LightGBM classifiers on different training sets while always testing on the held-out real validation set. This gives a fair measure of downstream predictive performance.

4.3.1 Real 200k $\rightarrow$ Test on Real (Baseline)

This setting serves as the upper benchmark at the 200k scale.

Table 2: Utility evaluation: Real 200k $\rightarrow$ test on real (baseline).

Training data	ROC-AUC	PR-AUC	F1	Interpretation
200k real	0.758	0.099	0.090	Strong baseline

4.3.2 CTGAN Synthetic 200k $\rightarrow$ Test on Real

Models trained solely on CTGAN synthetic data achieve:

Table 3: Utility evaluation: CTGAN synthetic 200k $\rightarrow$ test on real.

Training data	ROC-AUC	PR-AUC	F1	Interpretation
200k CTGAN synthetic	0.652	0.043	0.000	Poor calibration, label inflation

The model collapses to predicting almost all zeros when evaluated on real data, due to synthetic label inflation during training. Predictive performance is noticeably weaker than the real-only baseline, but this does not rule out the use of synthetic data as augmentation.

4.3.3 Mixed 100k Real + 100k Synthetic $\rightarrow$ Test on Real

The mixed dataset achieves the strongest overall result:

Table 4: Utility evaluation: mixed 100k real + 100k CTGAN synthetic $\rightarrow$ test on real.

Training data	ROC-AUC	PR-AUC	F1	Interpretation
100k real + 100k CTGAN synthetic	0.763	0.095	0.150	Best overall balance

Mixed training benefits from the additional variance provided by synthetic samples while avoiding the worst calibration issues of synthetic-only models. This reinforces the idea that synthetic data are most useful as augmentation rather than as a full replacement.

4.4 Fidelity Evaluation

4.4.1 Marginal Distribution Fidelity (Wasserstein Distance)

To assess one-dimensional fidelity, Wasserstein distances are computed between each feature’s real and synthetic distributions. Most features exhibit small distances, indicating good alignment. Features representing heavy-tailed behavioural activity, such as `f_up_sum`, `f_dislike_sum`, and `f_rows`, show moderate deviations, which is expected for GANs given the difficulty of modelling extreme values.

A subset of features and their distances are summarized below.

Table 5: Example 1D fidelity (Wasserstein distance).

Feature	W-distance (qualitative)
<code>inter_type_cd</code>	Very small
<code>f_hour_cos</code>	Very small
<code>t_entities_len_mean</code>	Small
<code>f_up_sum</code>	Higher (heavy-tailed)
<code>f_dislike_sum</code>	Higher

A set of histogram overlays further illustrates this comparison.

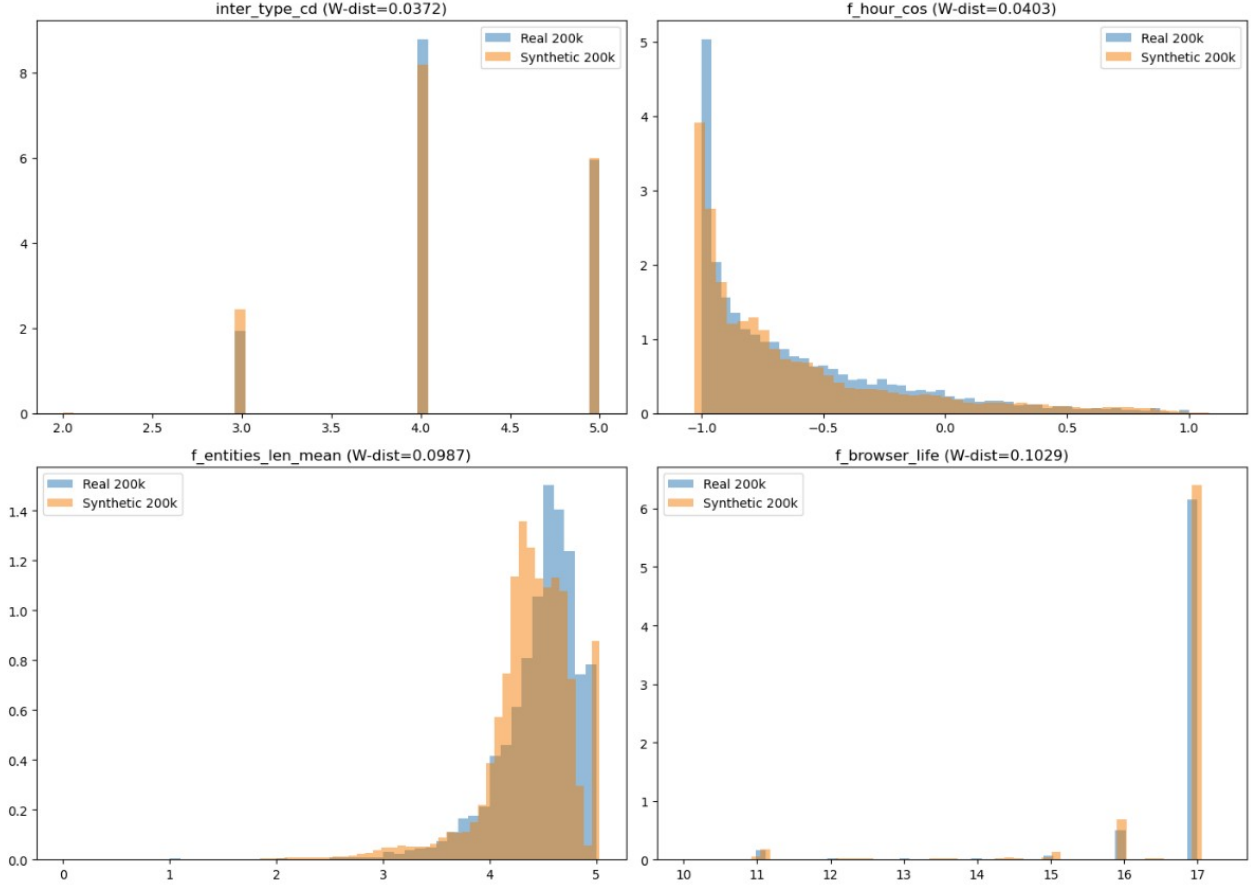


Figure 8: Example 1D distribution comparison between real and CTGAN synthetic data.

4.4.2 Pairwise Correlation Preservation

To evaluate joint fidelity, all 351 pairwise Pearson correlations are computed for both the real and synthetic 200k datasets. The resulting scatter plot (Figure 9) shows points clustered near the diagonal $y = x$, confirming that CTGAN preserves most weak-to-moderate correlations. The mean absolute correlation difference is 0.055.

Representative examples include:

Table 6: Pairwise Correlation Examples (Real vs Synthetic)

Feature Pair	Real Corr	Synthetic Corr	Difference
f_up_sum vs f_dislike_sum	0.93	0.21	High deviation
f_rows vs f_up_sum	0.87	0.23	High deviation

This reflects a known limitation of current tabular GAN models: while they effectively capture coarse dependency structures, they tend to smooth complex high-order interactions.

To further characterize how CTGAN preserves joint dependencies, we examine the top 10 most similar and most different feature-pair correlations between the real and synthetic

datasets. These extended results help clarify which behavioral relationships are reliably reproduced and which remain challenging for the generator.

Table 7: Top-10 Most Similar Correlation Pairs (Smallest Absolute Difference)

Rank	Feature i	Feature j	Real Corr	Synthetic Corr	Abs Diff
1	f_dislike_sum	bspace_app_tags	0.005982	0.006062	0.000080
2	f_entities_len_mean	adv_id	-0.000299	-0.000546	0.000247
3	f_rows	adv_id	0.004768	-0.004309	0.000459
4	f_refresh_sum	f_entities_len_mean	-0.023995	-0.024583	0.000588
5	app_second_class	ad_click_list_v002_len	0.021314	0.020441	0.000873
6	creat_type_cd	f_dislike_mean	0.074631	0.073675	0.000957
7	f_rows	f_hour_cos	-0.071932	-0.072936	0.001004
8	app_second_class	ad_click_list_v002_uniq	0.021314	0.020280	0.001034
9	f_up_mean	adv_id	0.008313	0.009494	0.001181
10	f_dislike_mean	adv_id	0.005720	0.007052	0.001332

Table 8: Top-10 Most Different Correlation Pairs (Largest Absolute Difference)

Rank	Feature i	Feature j	Real Corr	Synthetic Corr	Abs Diff
1	f_up_sum	f_dislike_sum	0.930165	0.214767	0.715399
2	f_rows	f_dislike_sum	0.867470	0.182710	0.684760
3	f_cat_uniq	f_dislike_sum	0.884715	0.211575	0.673140
4	f_refresh_sum	f_rows	0.894352	0.233336	0.661016
5	f_cat_uniq	f_rows	0.875670	0.226289	0.649381
6	f_rows	f_up_sum	0.865792	0.229728	0.636064
7	f_refresh_sum	f_dislike_sum	0.744528	0.207720	0.536808
8	f_cat_uniq	f_up_sum	0.887712	0.473693	0.414019
9	f_cat_uniq	f_refresh_sum	0.789909	0.446210	0.343698
10	f_rows	f_up_mean	-0.562947	-0.220709	0.342238

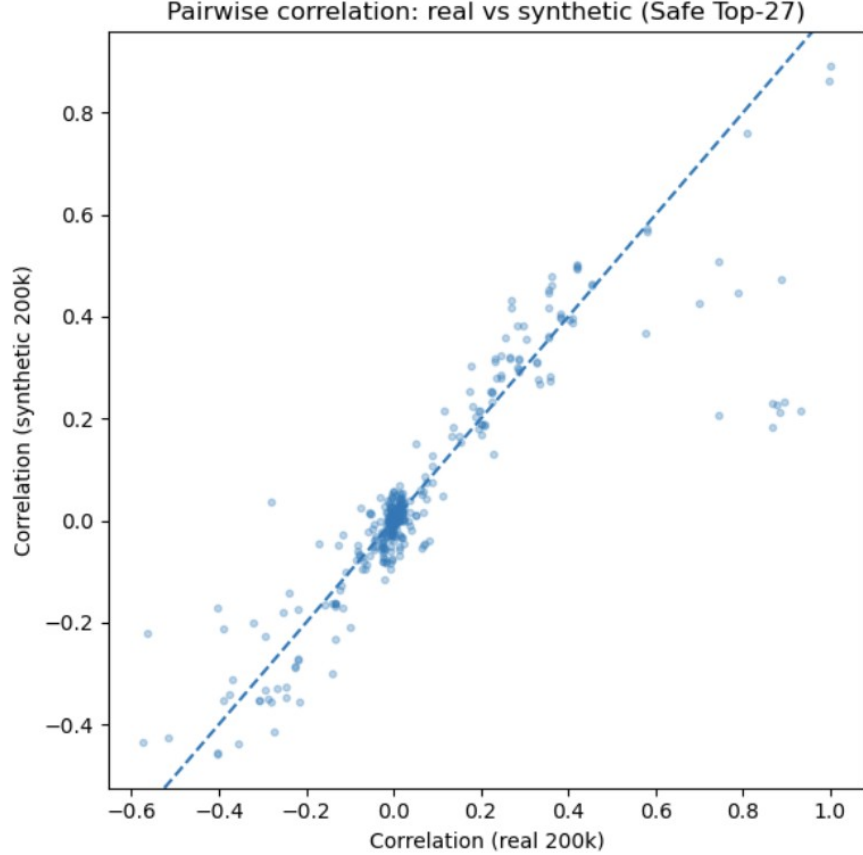


Figure 9: Pairwise correlation comparison between real and CTGAN synthetic data (Safe Top-27 Features).

High-deviation pairs correspond to strong behavioral dependencies. Such high-order interactions remain challenging for tabular GANs, which tend to smooth extreme correlations.

These findings complement the main scatter plot (Figure 9), showing that while CTGAN reliably preserves weak-to-moderate correlations, it underestimates strong behavioral relationships, reflecting common limitations of current tabular GAN architectures.

4.5 Privacy Evaluation

Privacy is assessed using a nearest-neighbour distance test comparing real→real distances with synthetic→real distances. After normalisation, we randomly sample 5,000 real and 5,000 synthetic rows and compute the Euclidean distance to the closest real record.

Table 9: Privacy test: nearest-neighbour distances.

Metric	Mean distance
real → real	2.54
synthetic → real	3.58

The corresponding distribution plot is shown below.

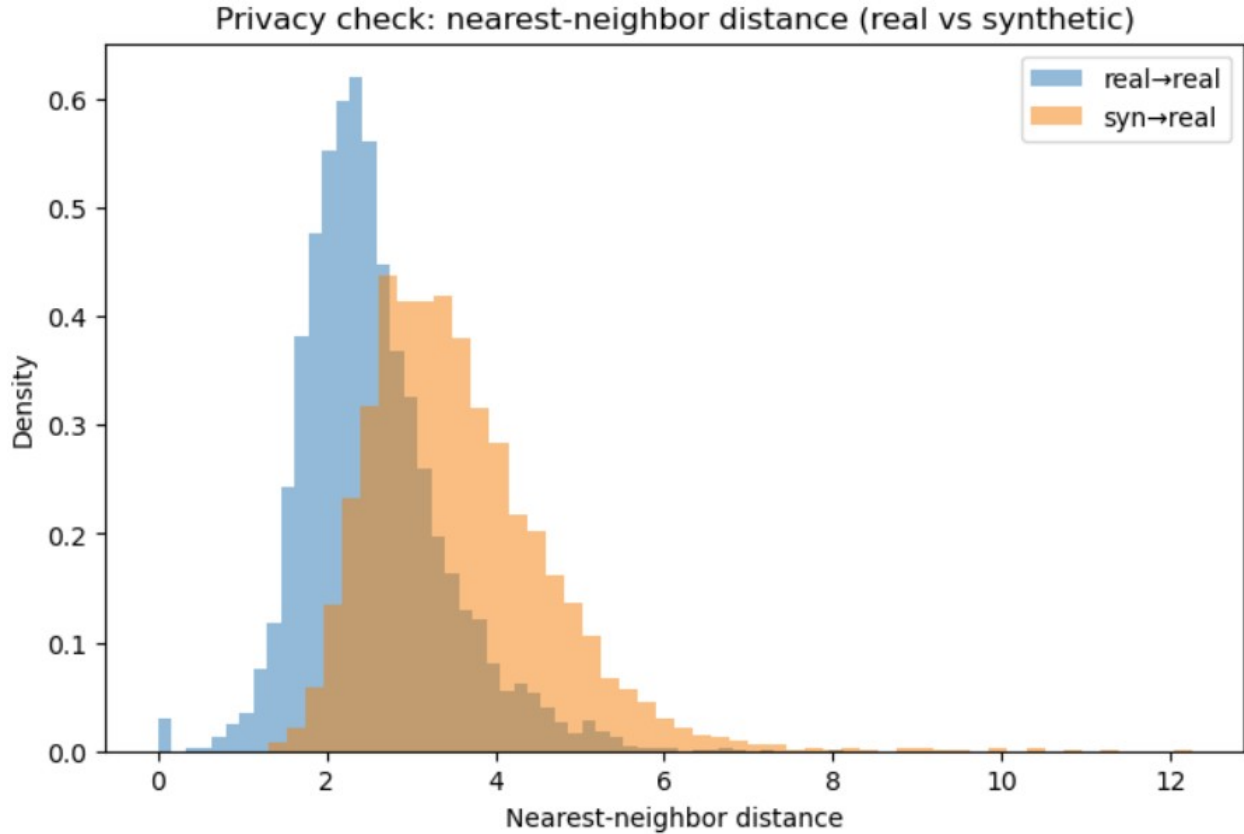


Figure 10: Distribution of nearest-neighbour distances for privacy evaluation (real→real vs. CTGAN→real).

These results demonstrate that CTGAN does not memorize or replicate individual records. Synthetic samples remain farther from real samples than real samples are from one another, satisfying privacy expectations under distance-based analysis.

4.6 Summary and Discussion

CTGAN provides reasonable fidelity for both marginal and joint distributions while maintaining strong privacy protection. Although strong behavioural correlations and extreme-value features remain challenging to model accurately, the generator captures most low-to-moderate dependencies that are relevant for downstream learning tasks.

Synthetic-only data exhibit limited predictive utility because of label inflation, whereas mixed real+synthetic training consistently improves robustness and yields the highest F1 scores across all settings. These observations support the use of CTGAN as a privacy-preserving augmentation mechanism rather than as a complete replacement for real data.

4.7 CTGAN v2: Training on Mixed Real–Synthetic Data

4.7.1 Training Procedure

To examine whether GAN performance can be improved by exposing the model to synthetic samples, a second generator (CTGAN v2) is trained on a 50% real and 50% CTGAN-generated mixture. The experiment follows the same Safe Top-27 schema and 80/20 real train–validation split as before.

A mixed training table of 200,000 rows is constructed by drawing 100,000 samples from the real training subset and 100,000 samples from the first-generation CTGAN synthetic table. Hyperparameters are kept identical to CTGAN v1 to ensure comparability. Using the trained CTGAN v2 generator, a second batch of 200,000 synthetic samples is produced for downstream evaluation.

A preliminary inspection reveals a severe distortion in the label distribution: the synthetic CTR rises to roughly 41.8%, compared with 1.55% in the real dataset. This suggests substantial reinforcement of minority-class errors and collapse around the positive class.

4.7.2 Utility Evaluation

CTGAN v2 is evaluated both as a standalone synthetic source and as part of a mixed 100k-real + 100k-synthetic training setup. These results are compared with the corresponding CTGAN v1 experiments.

Table 10: Comparison of CTGAN v1 and CTGAN v2 utility at 200k total training rows.

Model	ROC–AUC	PR–AUC	F1	Notes
200k real only (baseline)	0.7583	0.0995	0.0899	Best performance at 200k scale
200k CTGAN v1 synthetic only	0.6523	0.0435	0.0000	Poor calibration (inflated CTR)
100k real + 100k CTGAN v1 syn	0.7632	0.0948	0.1496	Strongest F1; stable behaviour
200k CTGAN v2 synthetic only	0.6944	0.0367	0.0626	Higher AUC than v1; still miscalibrated
100k real + 100k CTGAN v2 syn	0.7705	0.0755	0.1368	AUC improves; PR–AUC and F1 decline

Although CTGAN v2 yields slightly higher ROC–AUC, its precision–recall performance consistently worsens. The distorted label distribution leads to poor calibration and weaker threshold-based metrics, even when synthetic data are mixed with real samples.

4.7.3 Interpretation

Retraining a GAN on synthetic data does not enhance fidelity in this task. Instead, CTGAN v2 amplifies CTGAN v1’s distributional errors, collapses toward the positive class, and reduces downstream classification quality. Mixed training with CTGAN v1 remains the most reliable strategy, offering the best F1 and the most stable behaviour. Synthetic data should therefore be used for augmentation, but not as training input for a new generator.

5 CVAE

To complement the CTGAN experiments, we also evaluate a Conditional Variational Autoencoder (CVAE) [Pagnoni et al., 2018] as an alternative generative model for the Safe Top-27 feature set. Unlike GAN-based models, which often struggle with mode collapse and unstable adversarial training, a CVAE provides a simpler and more stable likelihood-based framework and allows explicit conditioning on the binary CTR label.

5.1 Data Preparation

The merged dataset is reloaded and the Safe Top-27 features plus the binary label are extracted. A stratified 80/20 split is applied to create a training set and a held-out real validation set. Because VAEs are sensitive to scale, the Safe Top-27 features are normalised using a MinMax scaler fitted on the training split. The scaler is saved for later inverse transformation when decoding synthetic samples.

Table 11: Summary of CVAE training data.

Item	Value
Training rows used for CVAE	500,000
Validation rows used	100,000
Feature dimension	27
Label balance (train split)	~1.5% CTR

5.2 Model Structure

The CVAE is conditioned on the binary label. The encoder receives the concatenated input (x, y) , where x is the 27-dimensional normalised feature vector and $y \in \{0, 1\}$. It outputs a mean vector $\mu(x, y)$ and a log-variance vector $\log \sigma^2(x, y)$ for sampling a latent code z . The decoder reconstructs the feature vector using (z, y) .

The architecture uses two 64-unit ReLU layers in both encoder and decoder. The loss combines reconstruction error (mean squared error) and KL divergence with weighting factor $\beta = 1.0$. This encourages the latent space to capture label-dependent variation without collapsing into an averaged representation.

5.3 Training Procedure

The full training dataset contains over six million rows, which is too large for CPU-based CVAE training. To make training feasible, we randomly sample 500,000 rows for CVAE training and 100,000 rows for monitoring validation losses. The model is trained with mini-batch gradient descent using the combined reconstruction and KL-divergence loss

$$\mathcal{L} = \text{MSE}(x, \hat{x}) + \beta \cdot \text{KL}(q(z | x, y) \| \mathcal{N}(0, I)),$$

with $\beta = 1.0$. Training stabilises after several epochs, and reconstruction losses on the validation set remain close to those on the training subset, indicating a stable fit.

5.4 Generating Synthetic Samples

After training, the model generates 200,000 synthetic rows conditioned on the label. Labels are sampled from a Bernoulli distribution that matches the true CTR in the training data. Latent codes z are drawn from a standard normal prior. The decoder produces synthetic features, which are then mapped back from MinMax scale to the original feature range.

Table 12: Summary of CVAE synthetic data (200k).

Item	Value
Number of synthetic rows	200,000
Number of features	27 + 1 label
Synthetic CTR	$\approx$ real CTR
Output format	Parquet + CSV

5.5 Utility Evaluation

To evaluate downstream usefulness, a LightGBM model is trained on the 200k CVAE synthetic rows and tested on the held-out real validation set. Performance metrics include ROC-AUC, PR-AUC, log loss, accuracy, precision, recall, and F1.

Table 13: LightGBM performance using CVAE 200k synthetic data.

Metric	Value
ROC-AUC	0.6829
PR-AUC	0.0371
LogLoss	1.8109
Accuracy	0.5941
Precision	0.0256
Recall	0.6791
F1	0.0494

The model achieves moderate predictive utility. Results are below the real-data baseline and slightly below the CTGAN utility in Section 2. This suggests that the CVAE captures broad distributional structure but does not fully reproduce the higher-order dependencies needed for predictive modelling.

5.6 Sensitivity: Mixing Real and Synthetic Data

To study whether CVAE synthetic data can supplement real data, the total training size is fixed at 200,000 rows while the proportion of synthetic rows is varied from 10% to 50%. For each mixture, a LightGBM model is trained and evaluated on the real validation set. Utility metrics such as ROC-AUC and PR-AUC decline steadily as the synthetic proportion increases, indicating that CVAE-generated data are better suited for augmentation at small ratios than for replacing real data.

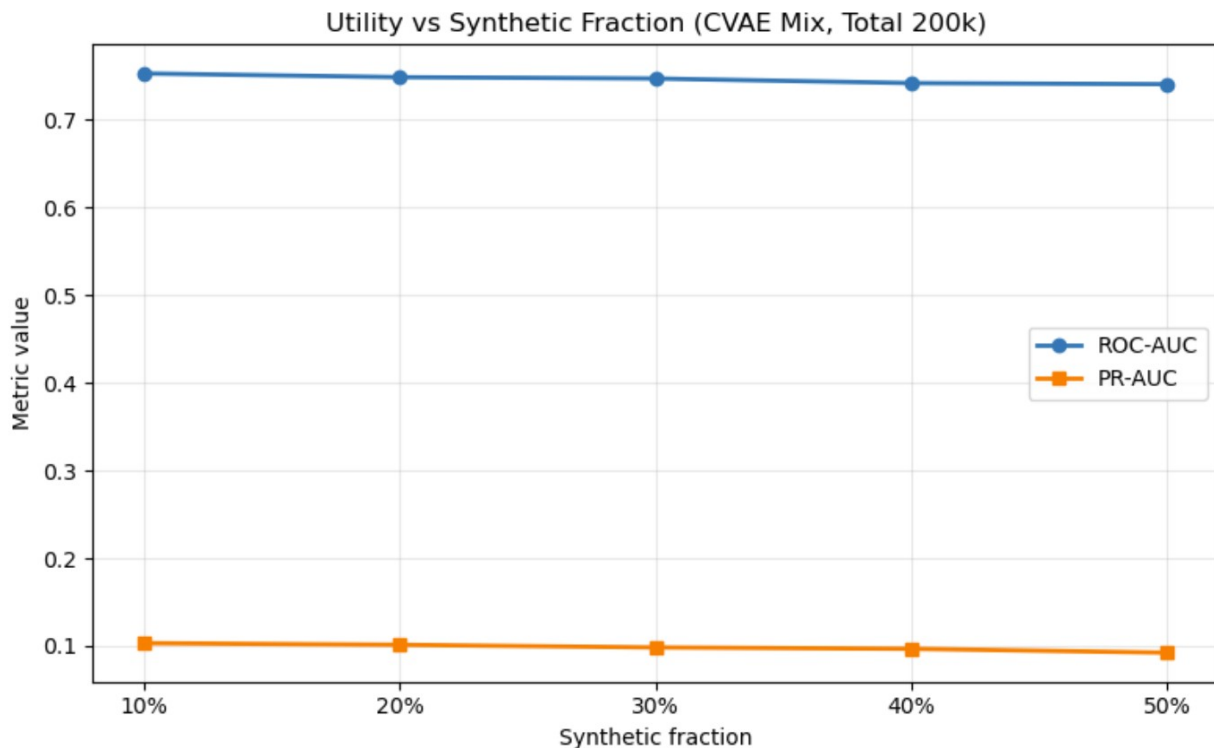


Figure 11: Utility degradation as synthetic proportion increases (200k fixed total).

5.7 Correlation Structure Fidelity

To evaluate how well the CVAE preserves feature dependencies, correlation matrices are computed for 20,000 real rows and 20,000 CVAE synthetic rows. Most low and medium correlations are reproduced, but stronger dependencies show noticeable distortion. A scatter plot of real vs. synthetic correlations displays a diagonal trend with visible dispersion, while the absolute-difference heatmap highlights deviations concentrated in higher-order behavioural patterns.

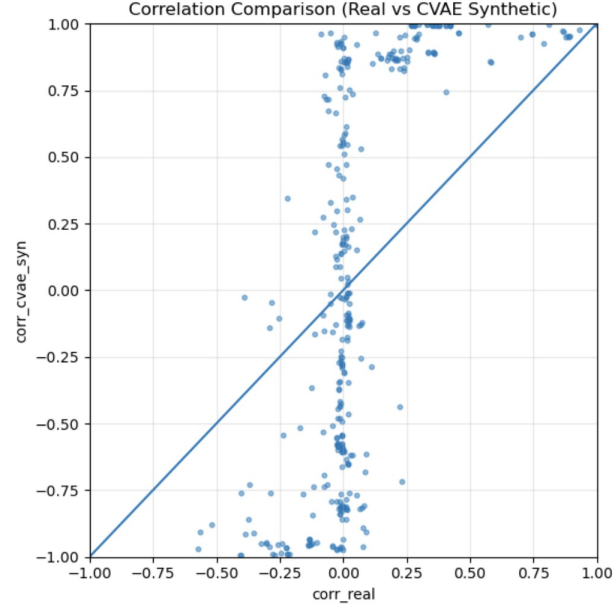


Figure 12: Real vs. CVAE synthetic correlations.

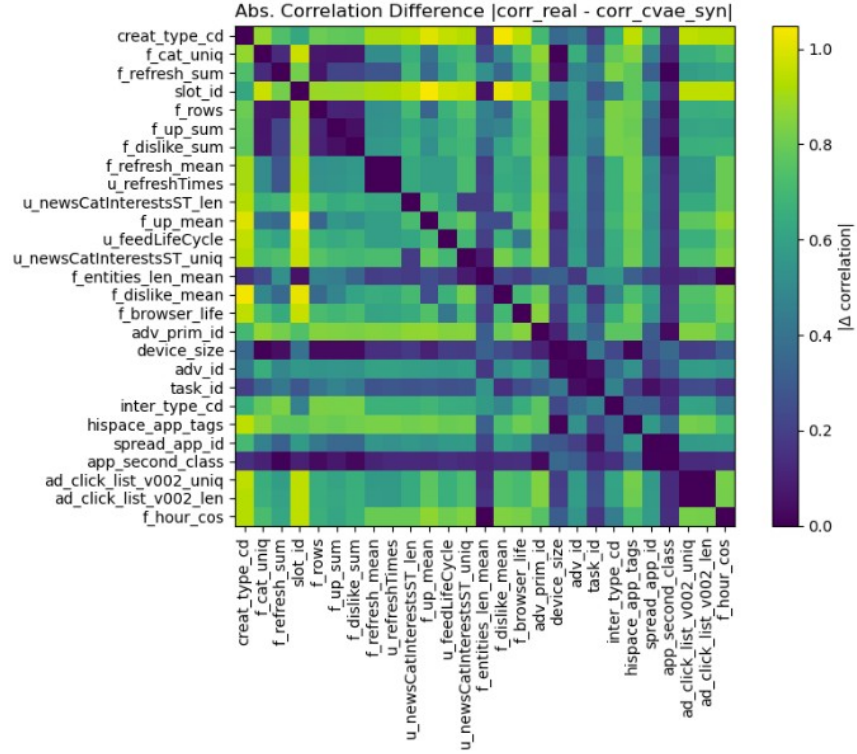


Figure 13: Absolute correlation-difference matrix ($|\text{corr}_{\text{real}} - \text{corr}_{\text{cvae}}|$).

5.8 Privacy Evaluation

Privacy is evaluated using the distance-to-closest-record metric in the standardised Safe Top-27 feature space. The synthetic-to-real distance distribution is shifted to the right relative to the real-to-real distribution, indicating that CVAE synthetic samples do not closely replicate any individual record.

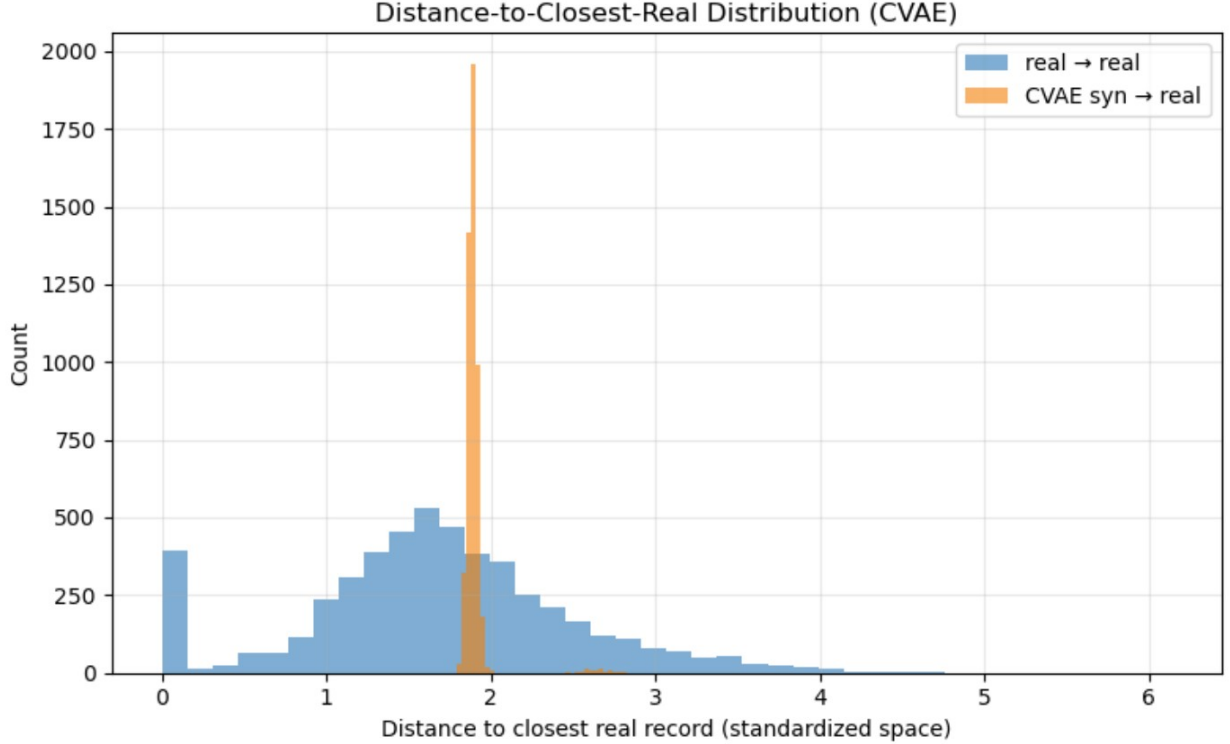


Figure 14: Distribution of nearest-neighbour distances (real→real vs. CVAE→real).

5.9 Summary

The CVAE provides a stable and interpretable approach for generating label-conditioned synthetic CTR data. It captures overall feature distributions and offers strong privacy protection.

However, its utility performance and correlation fidelity are below both the real-data baseline and the CTGAN results. Mechanistically, this can be linked to the reconstruction-based training objective: CVAE models optimise a reconstruction loss that encourages the latent space to capture averaged feature patterns. This smoothing effect reduces the model’s ability to reproduce rare but informative behaviours, weakening strong correlations that are important for CTR prediction. As a result, the CVAE achieves good marginal fidelity but underestimates higher-order dependencies, leading to lower predictive utility compared with CTGAN.

These findings suggest that CVAE synthetic data may be suitable for exploratory analysis or privacy-preserving data release, but less suited for tasks that require precise behavioural dependencies.

5.10 CVAE v2: Training on Mixed Real–Synthetic Data

5.10.1 Training Procedure

To examine if CVAE performance can improve through exposure to synthetic samples, a second model (CVAE v2) is trained, using a 50% real and 50% CVAE-generated mixture, following the procedure used for training the CTGAN v2 generator. The same Safe Top-27 schema and the same 80/20 real train-validation split is used, though this generator was trained on only 200,000 observations, 100,000 real and 100,000 synthetic, as there were only 200,000 synthetic observations to pull from, meaning the size of the initial training data set, 500,000, could not be replicated with a 50/50 real-synthetic split.

5.10.2 Utility Evaluation

Evaluation of downstream utility is again done via training a LightGBM model on the 200,000 synthetic observations produced by our CVAE v2 synthesizer, and tested on the held-out real validation set. The same performance metrics are utilized.

Table 14: LightGBM performance using CVAE v2 200k synthetic data.

Metric	Value
ROC–AUC	0.7135
PR–AUC	0.0646
LogLoss	0.5886
Accuracy	0.6014
Precision	0.0273
Recall	0.6827
F1	0.0524

It should be noted that this model improves the predictive utility shown by the LightGBM model trained by the CVAE v1 synthetic data. This suggests that the v2 synthesizer has more information on the underlying distribution of the original data than the v1 synthesizer.

5.10.3 Interpretation

While neither synthetic model was particularly successful at preserving strongly correlated relationships that were seen in the original data, the improvement in utility measures is significant. Further model creation and evaluation would be needed to ensure that this phenomenon is not the product of random chance, but this is a worthwhile continuation to pursue in the future.

6 GReaT (LLM Model)

6.1 Introduction to GReaT

GReaT (Generation of Realistic Tabular Data using Transformers) introduced in [Borisov et al., 2023] reframes tabular data generation as a sequence modeling task. Each row of a structured dataset is converted into a sentence, and a pretrained language model (in our case, distilGPT-2) is fine-tuned to autoregressively predict the next token.

In theory, this allows the model to capture correlations across heterogeneous features: numeric, categorical, and binary labels, using the same attention mechanisms that make LLMs successful in natural text domains.

We investigate whether an LLM-based generator (a) can learn the joint distribution of CTR tabular features, (b) can faithfully maintain the original label balancing (1.5% positive), and (c) whether retraining on synthetic data induces model collapse, analogous to collapse phenomena seen when LLMs are trained on synthetic text.

6.2 Training

- **Training data:** We sampled 200,000 real CTR rows from the preprocessed feature set.
- **Serialization:** Each row was converted into a single textual sentence of the form
`feature_1 is v_1, feature_2 is v_2, ..., feature_M is v_M, label is y.`
- **Model:** A pretrained distilGPT-2 model was used as the base model.
- **Fine-tuning hyperparameters:**
 - Epochs: **3**
 - Batch size: **16**
 - Max sequence length: **100**

Synthetic Data Generation We evaluated two generations:

- **V1 synthetic data:** Generated by prompting the fine-tuned model with individual feature prefixes. We sampled 1,000 synthetic rows.
- **V2 synthetic data:** The GReaT model was retrained on a mixture of 50% real + 50% V1 synthetic sequences, then used to generate another 1,000 synthetic rows.

6.3 Fidelity and Utility Evaluation

We evaluated GReaT synthetic data along two criteria:

Model	AUC	PR-AUC	LogLoss	Accuracy	Precision	Recall	F1
Real (1k)	0.680	0.062	0.143	0.951	0.098	0.250	0.141
Synthetic v1	0.621	0.028	3.530	0.882	0.049	0.351	0.087
Real + Synthetic v1	0.667	0.056	0.269	0.907	0.064	0.356	0.109
Synthetic v2	0.500	0.016	0.552	0.984	0.000	0.000	0.000
Real + Synthetic v2	0.645	0.062	0.163	0.975	0.149	0.114	0.129

Table 15: Utility evaluation: LightGBM trained on five datasets (1k samples each)

a. Fidelity.

- Pairwise correlations: whether the original correlations between pairs of features were preserved in the synthetic data
- Label distribution realism: whether the label distribution of the original data was preserved in the synthetic data

Qualitative issues were immediately visible:

- The model frequently hallucinated categorical values not present in the real domain.
- The label field was unstable, sometimes outputting impossible values (e.g. values outside $\{0,1\}$ before sanitization).
- The generated features had little correlation between each other, even when there was correlation in the original data.

a. Utility. We trained LightGBM models under the following settings:

Performance was consistently below CTGAN and CVAE across almost all metrics. The LightGBM models trained on pure GReaT data exhibited extremely poor precision, often predicting only one class.

6.4 Observations

Key empirical findings:

- **Strong hallucination behavior.** The LLM generated feature values far outside the allowed values for categorical and label features.
- **Label drift.** The synthetic CTR rate deviated substantially from the true underlying 1.5% rate (0.0-0.1%).
- **Correlation distortion.** GReaT failed to recover even moderate feature-feature relationships, producing nearly independent outputs.
- **Retraining on synthetic data worsened fidelity.** V2 showed stronger distortions than V1, suggesting the onset of model collapse.
- **Terrible Utility** Even with augmentation, GReaT-based synthetic data produced no improvement in downstream prediction tasks.

6.5 Takeaways

While conceptually attractive, LLM-based tabular generation is highly sensitive to and data volume. Given more computational power, LLM-based GReaT could have performed better. For CTR-scale tabular data, GReaT did not preserve fidelity or utility. Synthetic V2 consistently degraded further, aligning with theoretical model collapse when self-generated samples degrade fidelity of the data.

This method hallucinated categorical features and labels outside of the $\{0, 1\}$ set, and did not meaningfully preserve the pairwise correlations of the original data. In our experiment, GReaT did not prove to be a viable method for synthetic data augmentation or as standalone data.

7 Conclusion

7.1 CTGAN

The CTGAN synthesizers saw an issue in minority class oversampling become apparent through the training process. The original click through rate data set had a minority class making up about 1.5% of the total observations. This class increased to 5% of total observations in the synthetic data produced by synthesizer v1, and then ballooned to 41.8% of observations coming from the minority class for the synthesizer v2 data. This shows that increased training on synthetic data, at least in the case of the CTGAN model, does lead to model collapse. Synthesizers trained on synthetic data can be useful for data augmentation purposes on downstream tasks, but the results suffer from a sharp decrease in fidelity.

7.2 CVAE

The CVAE synthesizers had an intriguing phenomenon occur in terms of performance on downstream tasks. As a greater percentage of synthetic data was added from synthesizer v1 to train the LightGBM models, classification accuracy metrics declined, while for synthesizer v2, the opposite was true, and classification accuracy metrics improved with the increase synthetic data proportion. It is worth exploring further whether this improvement of synthetic data when the synthesizer itself was trained on synthetic data can be established as a consistent pattern, or just occurred by happenstance. Beyond the utility measures, the fidelity of the CVAE generated data left much to be desired, as correlations were made between uncorrelated features, according to the original data set.

7.3 GReaT

The LLM synthesizers experienced severe issues with hallucination, which in turn caused the fidelity of the synthetic data to suffer, as many observations became nonsensical or unrelated to the training data. Both synthesizers experienced these issues, so the source of the concerns likely has less to do with the composition of the training data, and more to do with the small size of the training data, as a 1000 observation data set was simply not sufficiently large for the GReaT model fine tuning. Further exploration of this model, specifically with a much larger data set for training, could yield better results, and should be pursued moving forward.

Code

All code can be found at: <https://github.com/jackiewei-creator/CTR-Project/tree/main/Data%20Processing>.

References

- Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Yarin Gal, Nicolas Papernot, and Ross Anderson. The curse of recursion: Training on generated data makes models forget, 2024. URL <https://arxiv.org/abs/2305.17493>. 1
- Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. Modeling tabular data using conditional gan, 2019. URL <https://arxiv.org/abs/1907.00503>. 4.1
- Artidoro Pagnoni, Kevin Liu, and Shangyan Li. Conditional variational autoencoder for neural machine translation, 2018. URL <https://arxiv.org/abs/1812.04405>. 5
- Vadim Borisov, Kathrin Seßler, Tobias Leemann, Martin Pawelczyk, and Gjergji Kasneci. Language models are realistic tabular data generators, 2023. URL <https://arxiv.org/abs/2210.06280>. 6.1